TECHNICAL REPORT: ADVANCED PREDICTIVE MODELING FOR REGIONAL SALES PERFORMANCE AND DEMAND FORECASTING

Name: Susan Wanjiru
StudentID: 673011
Course Code: DSA2050
Primary Dataset: `DSA2050_Week6_Sales_Data.csv`
Execution Environment: Python 3.13.15 | scikit-learn | Pandas | NumPy

---

EXECUTIVE SUMMARY

This analytical report evaluates retail sales performance across multiple regions, customer segments, and product categories using linear, regularized, and tree-based regression methodologies. The business objective is to transition from heuristic sales forecasting to an automated, statistically validated machine learning pipeline that predicts gross revenue (`Sales_KES`) based on advertising budgets, store traffic, discount structures, regional footprints, and product verticals.

Key Performance Findings

1. **Model Superiority:** The **Ridge Regression** model (L2 regularization, $\alpha = 1.0$) achieved the highest out-of-sample predictive accuracy, yielding a Test Mean Absolute Error (**MAE**) of **13,714.40 KES**, Root Mean Squared Error (**RMSE**) of **17,458.82 KES**, and an R^2 **score of 0.8705** (explaining ~87.05% of sales variance).
2. **Baseline Comparison:** Naïve baseline strategies (Mean/Median Dummy Regressors) produced negative R^2 scores (~ -0.0056 to -0.0084), demonstrating that simple central-tendency estimators fail to capture structural variation in customer volume and advertising response.
3. **Overfitting Mitigation:** Non-linear Decision Tree models exhibited significant sub-optimality ($R^2 = 0.6484$, Test MAE = 23,775.92 KES) due to sample size constraints ($N = 320$) and step-function variance, confirming linear models as the preferred approach for this feature space.
4. **Generalization Stability:** 5-fold cross-validation confirmed consistent performance for Ridge Regression (`$\text{CV MAE} = 15,971.37\text{ KES}$`, `$\text{CV R^2 = 0.8479$`), verifying model stability against cross-validation folds.

1. PROJECT OVERVIEW & SCOPE

1.1 Problem Statement

Retail management requires accurate sales prediction to optimize operational workflows, including inventory allocation, regional advertising spending, and targeted promotional discounting. Forecasting sales in Kenyan Shillings (KES) allows financial teams to set precise revenue targets and prevent over/under-stocking across regional distribution centers.

1.2 Data Infrastructure & Schema

The dataset comprises $N = 320$ historical store records across seven operational attributes. Data integrity verification confirmed zero null entries and zero duplicate rows.

Column Name	Data Type	Role	Summary Statistics / Unique Values
`AdvertisingSpend_KES`	Continuous (int64)	Feature	Min: 5,478 | Mean: 37,421.78 | Max: 69,746
`StoreVisits`	Continuous (int64)	Feature	Min: 84 | Mean: 465.44 | Max: 848
`DiscountPercent`	Continuous (int64)	Feature	Min: 0% | Mean: 12.81% | Max: 25%
`Region`	Nominal (object)	Feature	4 Unique: Nairobi (139), Coast, Rift Valley, Western
`CustomerSegment`	Nominal (object)	Feature	3 Unique: Retail (157), Wholesale, Corporate
`ProductCategory`	Nominal (object)	Feature	4 Unique: Electronics (95), Apparel, Home, Groceries
`Sales_KES`	Continuous (int64)	Target (y)	Min: 17,805 | Mean: 169,252.45 | Max: 295,825

2. EXPLORATORY DATA ANALYSIS (EDA) & PREPROCESSING

2.1 Distribution Analysis

- **Target Variable (`Sales_KES`):** Standard Deviation = 51,968.08 KES. Median = 168,462.00 KES against Mean = 169,252.45 KES, demonstrating a near-symmetric Gaussian-like distribution suitable for squared-error loss minimization without explicit logarithmic transformations.
- **Predominant Categories:** Nairobi represents 43.4% of regional entries ($n = 139$), Retail forms 49.1% of customer segments ($n = 157$), and Electronics is the top product category ($n = 95$).

2.2 Feature Engineering & Pipeline Construction

To eliminate data leakage and maintain production readiness, feature transformation was wrapped inside an automated `ColumnTransformer` scikit-learn assembly:

1. **Numerical Pipeline:**
   - Median Imputation (`SimpleImputer(strategy='median')`)
   - Z-score Standard Scaling (`StandardScaler()`): Transforms features to $\mu = 0, \sigma = 1$ to equalize gradient penalties for regularized models.
2. **Categorical Pipeline:**
   - Most Frequent Imputation (`SimpleImputer(strategy='most_frequent')`)
   - One-Hot Encoding (`OneHotEncoder(handle_unknown='ignore')`): Encodes nominal variables into binary indicator vectors while dropping unknown categories gracefully during inference.

---

3. MODEL ARCHITECTURE & REGRESSION FRAMEWORK

Five distinct modeling strategies were developed and tested using an 80/20 train-test split (`$N_{\text{train}} = 256$`, `$N_{\text{test}} = 64$`) with a fixed random seed (`RANDOM_STATE = 42`).

3.1 Evaluated Estimators

1. **Mean Baseline Regressor:** Predicts `$\hat{y} = \bar{y}_{\text{train}}$` for all samples.
2. **Median Baseline Regressor:** Predicts `$\hat{y} = \text{median}(y_{\text{train}})$` for all samples.

3. **Ordinary Least Squares (OLS) Linear Regression:** Fits unregularized coefficients via normal equations.
4. **Ridge Regression:** Applies L2 Tikhonov regularization penalty ($\alpha \sum \beta_j^2$) to limit coefficient variance.
5. **Decision Tree Regressor:** Non-parametric axis-aligned splitting model constrained to `max_depth=4` and `min_samples_leaf=8` to limit tree growth.

---

4. EXPERIMENTAL RESULTS & METRIC EVALUATION

4.1 Evaluation Metrics Definition

- **Mean Absolute Error (MAE):** $\frac{1}{n}\sum |y_i - \hat{y}_i|$ (Interpretable average KES error).
- **Root Mean Squared Error (RMSE):** $\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$ (Penalizes large outlier errors).
- **Coefficient of Determination (R^2):** $1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$ (Proportion of explained variance).

4.2 Test Set Performance Summary

Model Rank	Model Name	MAE (KES)	RMSE (KES)	R^2 Score
1	**Ridge Regression ($\alpha = 1.0$)**	**13,714.40**	**17,458.82**	**0.870488**
2	Linear Regression (OLS)	13,757.84	17,469.69	0.870327
3	Decision Tree (`max_depth=4`)	23,775.92	28,767.64	0.648368
4	Median Baseline	39,146.75	48,649.47	-0.005626
5	Mean Baseline	39,197.16	48,716.43	-0.008396

4.3 5-Fold Stratified Cross-Validation Results

To verify that the 80/20 test split results were not an artifact of random holdout sampling, 5-Fold Cross-Validation was run across the training set ($N = 256$).

Model Name	CV Mean MAE (KES)	CV Mean RMSE (KES)	CV Mean R^2
Ridge Regression	**15,971.37**	**20,103.54**	**0.847907**
Linear Regression	15,976.77	20,126.77	0.847517
Decision Tree	24,498.47	30,608.68	0.650627

---

5. DIAGNOSTICS & RESIDUAL ANALYSIS

5.1 Error Distribution Analysis

Analysis of linear residuals ($e_i = y_i - \hat{y}_i$) on the holdout test set revealed:

- **Mean Residual:** close to zero, validating unbiased estimations.
- **Homoscedasticity:** Residual plots against fitted values show consistent variance across low, medium, and high sales volumes, confirming that the linear formulation holds without requiring non-linear target transformations.
- **Outlier Sample Check:** A small subset of observations exhibited residuals larger than 35,000 KES (e.g., Row Index 3: Actual = 133,621 KES, Predicted = 172,495 KES, Residual = -38,874.44 KES), likely driven by unobserved promotional campaigns or local macroeconomic events.

---

6. BUSINESS RECOMMENDATIONS & DEPLOYMENT STRATEGY

1. **Deploy Ridge Pipeline into Production:** Deploy `ridge_model` (Pipeline combining `ColumnTransformer` and `Ridge(alpha=1.0)`) as the standard backend estimator for weekly store revenue forecasting.
2. **Prioritize Marketing & Foot Traffic Features:** Linear regression models benefit heavily from `AdvertisingSpend_KES` and `StoreVisits`. Management should align regional advertising budgets directly with predicted foot traffic thresholds.
3. **Continuous Monitoring:** Establish automated data quality checks for out-of-range discount inputs (`$>25\%$`) or unmapped regional categories to prevent feature shift during model inference.

```
None
+------------------------------------------------------------------------+
|                        PRODUCTION PIPELINE FLOW                        |
|                                                                        |
```

```
|  [ Raw Inputs ]                                                          |
|  (Spend, Visits, Discount, Region, Segment, Category)                    |
|         |                                                                |
|         ▼                                                                |
|  [ ColumnTransformer ]                                                   |
|  ├── Numeric: SimpleImputer(median) -> StandardScaler()                  |
|  └── Categorical: SimpleImputer(frequent) -> OneHotEncoder(ignore)       |
|         |                                                                |
|         ▼                                                                |
|  [ Ridge Regressor (alpha=1.0) ]                                         |
|         |                                                                |
|         ▼                                                                |
|  [ Predicted Sales (KES) ]                                               |
+--------------------------------------------------------------------------+
```

---

7. APPENDIX: REPRODUCIBILITY CODE SNIPPET

Python

```python
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load dataset
df = pd.read_csv("DSA2050_Week6_Sales_Data.csv")
X = df.drop(columns="Sales_KES")
y = df["Sales_KES"]

# Pipeline construction
num_cols = ["AdvertisingSpend_KES", "StoreVisits", "DiscountPercent"]
cat_cols = ["Region", "CustomerSegment", "ProductCategory"]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            Pipeline(
                [
                    ("imputer", SimpleImputer(strategy="median")),
                    ("scaler", StandardScaler()),
                ]
```

```
            ),
            num_cols,
        ),
        (
            "cat",
            Pipeline(
                [
                    ("imputer", SimpleImputer(strategy="most_frequent")),
                    ("onehot", OneHotEncoder(handle_unknown="ignore")),
                ]
            ),
            cat_cols,
        ),
    ]
)

ridge_pipeline = Pipeline(
    steps=[("preprocess", preprocessor), ("model", Ridge(alpha=1.0))]
)

# Model Training
ridge_pipeline.fit(X, y)
```